



Introduction to Programming

SSgt Clark, Athan L

Presented on 20251218

SSgt Clark

Platoon Sergeant

3D Network Battalion, Detachment Hawaii

- Software Developer for ~15 Years
- Software Engineering & Computer Science Enthusiast
 - Working Toward Degree and PE Licensure
- CompTIA A+, Net+, Sec+, CySA+
- Google Data Analytics, Professional Scrum Master Level 1
- Lean Six Sigma Yellow Belt

Outline

1. Introduction
2. Data and Actions
3. Basic Operations
4. Principle of Organization
5. Functions
6. Paradigms and Semantics
7. Composition of Data
8. Algebraic Data Types
9. Objects and OOP
10. Lambda Calculus
11. Type Theory
12. Conclusion

Introduction



“ Programming is the act of designing a system that runs on another system designed to run systems. ”

— Me

Data and Actions

Data = "Subject Matter"

Actions = "What I'm Doing With It"

```
digraph G {  
  rankdir=LR;  
  A -> B [label="F"];  
  A -> A [label="G"];  
}
```

General Ideas

- Data is inert
- Actions typically create data as a result

Basic Operations

Typically "Built-In" to the programming language

↪ a globally available "context" when designing a program

Data Types	Operations Supported
Numbers	Arithmetic, Trigonometry, Comparison
Booleans	Logic
Strings	Find-and-Replace, Concatenate/Append, "to lower" / "to upper"
Arrays	Same as strings except ambiguous, "Mapping", "Reduce"
Tuples	Mapping over Specific Position
Most	Equality

Principle of Organization

1. Doing a lot of different things can get complicated
2. Organize the actions into sub-actions, group data together
3. Make sure the sub-actions and groups are *"modular"*
4. Modular actions and data groups can now be used as built-ins

Programmers build tools to write programs, they extend and control their **context**

- Is not "endless" - there is an end-state of a "product"

Functions

Means different things in different programming paradigms

- Procedure captured over variables
- Generalized over inputs that remain ambiguous
- Can return results

```
square :: Number -> Number  
square x = x * x
```

```
function square(x: number) -> number {  
    return x * x;  
}
```


Paradigms and Semantics

- **"Mutable"** Data - variables that can have their contents mutated / changed
- **"Pure"** Functions - functions that don't cause side-effects
- **"Imperative"** vs. **"Functional"** - sequence of steps vs. composition of results
- **"Compiled"** vs. **"Interpreted"** - CPU executes the program via translation ahead of time, or just-in-time translation via an *interpreter*

Types of Data

- Object-Oriented, Mutable Objects that Contain Methods
- Algebraic Data Types, Separate Enumerations and Structures
- Inheritance, Traits, Generics - Means By Which Code Is Shared

Are We Good?

Are We Good?

Cool.

Install VSCode

code.visualstudio.com

Install Node.js

1. Install NVM-Windows: github.com/coreybutler/nvm-windows
2. Open PowerShell
3. Verify NVM is installed: `nvm`
4. Install a Long-Term Support Node.js: `nvm install lts`
5. Verify Node.js is installed: `node --version`

JavaScript

- Mutable
- Impure
- Imperative
- Interpreted
- Object-Oriented
- Inheritance

It's widely available (all browsers support it)

Composition of Data

- Data "Contained" Within other *Structures*
- Data that "Inherits" Functionality or Features

Typical Methods of Encapsulation

- Struct - Tuples
- Enum - "Either X or Y " - must be only 1
- " X is a Number" $\implies$ " X supports Arithmetic Operations"

Algebraic Data Types

Structure / Struct:

Rust:

```
struct Foo {  
    field_one: i32,  
    field_two: String,  
}
```

Haskell:

```
data Foo = Foo {  
    fieldOne :: Integer,  
    fieldTwo :: String,  
}
```

TypeScript:

```
type Foo = {  
    fieldOne: number,  
    fieldTwo: string,  
}
```

OOP:

```
class Foo {  
    int fieldOne;  
    char[] fieldTwo;  
}
```


Algebraic Data Types

Enumeration / Enum:

Rust:

```
enum Bar {  
    OnePossibility(i32),  
    OtherPossibility(String),  
    YetAnotherPossibility,  
}
```

Haskell:

```
data Bar  
    = OnePossibility Integer  
    | OtherPossibility String  
    | YetAnotherPossibility
```

A `Bar` must be either a `OnePossibility` holding a `i32` / `Integer`, a `OtherPossibility` holding a `String`, or a `YetAnotherPossibility`, which doesn't hold anything.

Why "Algebraic"?

- Struct = Product
- Enum = Sum

Think about the space of possible values a data type defines - a tuple or a struct multiplies the total number of possible values, while an enum just adds them together

- Example: Bit Depth - 2^n number of possibilities as n grows larger. If it's 2 bits, then it's 2×2

Why "Algebraic"?

```
data Binary = BZero | BOne -- total possible values = 2
data Trinary = TZero | TOne | TTwo -- total possible values = 3

data BothAtTheSameTime = BothAtTheSameTime Binary Trinary
data EitherOneOrTheOther = JustBinary Binary | JustTrinary Trinary
```

Both possible values	Count
BothAtTheSameTime BZero TZero	1
BothAtTheSameTime BZero TOne	2
BothAtTheSameTime BZero TTwo	3
BothAtTheSameTime BOne TZero	4
BothAtTheSameTime BOne TOne	5
BothAtTheSameTime BOne TTwo	6

Either possible values	Count
JustBinary BZero	1
JustBinary BOne	2
JustTrinary TZero	3
JustTrinary TOne	4
JustTrinary TTwo	5

We O.K.?

We O.K.?

Cool.

Play Around with the REPL

Showcase Basic Operations, Building Objects and Arrays, Methods on Objects

Show Some Basic Rust and Haskell Code

Object-Oriented Programming

- "Objects"
 - Contain Mutable Data (like a Struct)
 - Contain "**Methods**" (actions / functions) that affect its Data
- "**Classes**" Defines Types of Objects
- "**Instances**" Are the Objects Themselves

```
x = new Foo();  
x.runSomeMethod();  
subData = x.accessSomeField;
```

Lambda Calculus

Way of Writing Functions Short-Hand

$$\lambda x. x + 5$$

```
function add5(x) {  
  return x + 5;  
}
```

Higher-Order Functions

$$\lambda f x. f(x)$$

“ Give me a function and a value and I'll apply that function to the value

”

Lambda Calculus

Way of Writing Functions Short-Hand

$$\lambda x. x + 5$$

```
function add5(x) {  
  return x + 5;  
}
```

Higher-Order Functions

$$\lambda f x. f(x)$$

$$\lambda f x. f\ x$$

Why Lambda Calculus?

In λ , functions are treated as data - functions can be returned from functions, functions can be passed to functions

Many programming languages' `function` keyword aren't lambdas, but they do support lambdas in different ways:

- Rust and Ruby is `|x| x + 5`
- JavaScript is `x => x + 5`
- C++ is weird, idk how it works
- Haskell uses lambdas natively

Type Theory

A fancy way of asking and finding out "what data type is this?"

Rust:

```
fn foo(x: i32) -> bool {  
    //...  
}
```

TypeScript:

```
function foo(x: number) -> boolean {  
    // ...  
}
```

Type Theory

Untyped Lambda Calculus

$$\lambda x. x + 5$$

Simply Typed Lambda Calculus

$$\lambda x : \textit{Number}. x + 5$$

Type Theory

Untyped Lambda Calculus

$$\lambda x. x + 5$$

Simply Typed Lambda Calculus

$$\lambda x : \textit{Number}. x + 5$$

$$f := \lambda x : \textit{Number}. x + 5$$

Type Theory

Untyped Lambda Calculus

$$\lambda x. x + 5$$

Simply Typed Lambda Calculus

$$\lambda x : \textit{Number}. x + 5$$

$$(\lambda x : \textit{Number}. x + 5) : \textit{Number} \rightarrow \textit{Number}$$

$$f := \lambda x : \textit{Number}. x + 5$$

$$f : \textit{Number} \rightarrow \textit{Number}$$

Type Theory

Generic Types - Type Variables

$$g := \lambda x. x$$

What is the type of g ?

Type Theory

Generic Types - Type Variables

$$g := \lambda x : \alpha. x$$

What is the type of g ?

Type Theory

Generic Types - Type Variables

$$g := \lambda x : \alpha. x$$

What is the type of g ?

$$g : \alpha \rightarrow \alpha$$

Type Theory

Generic Types - Type Variables

$$g := \lambda x : \alpha. x$$

What is the type of g ?

$$g : \alpha \rightarrow \alpha$$

α is a *type variable*

Type Theory

Generic Types - Type Variables

$$g := \lambda x : \alpha. x$$

What is the type of g ?

$$g : \alpha \rightarrow \alpha$$

α is a *type variable* - doesn't that mean we should introduce it with a lambda?

Type Theory

Generic Types - Type Variables

$$g := \forall \alpha. \lambda x : \alpha. x$$

What is the type of g ?

$$g : \forall \alpha. \alpha \rightarrow \alpha$$

α is a *type variable* - doesn't that mean we should introduce it with a lambda?

Type Theory

Generic Types - Type Variables

$$g := \forall \alpha. \lambda x : \alpha. x$$

What is the type of g ?

$$g : \forall \alpha. \alpha \rightarrow \alpha$$

α is a *type variable* - doesn't that mean we should introduce it with a lambda?

```
g :: forall a. a -> a
g x = x
```

How are we holding up?

How are we holding up?

Cool.

Further Information

- There is so much further information, literally just google anything about the language you want to learn
- Almost every language has their documentation online for free
- People compete with each other like influencers to make information more available

Conclusion



“ Good programmers are good at writing programs. Great programmers are good at designing programs. The best programmers are mathematicians. ”

Slides are available at
github.com/athancclark/usmc-presentation-programming-20251218

Vote on Next Topic

1. Proxmox Virtualization System

- great for home labs

2. Abstract Algebra

- don't be afraid

3. Shells

- 

4. GNU/Linux

- This one won't get you anywhere, but you'll be able to say that you were made aware of some things

Questions / Comments